

1. Overview

Real-time video analytics are now common in domains such as traffic monitoring, surveillance, and smart cities. Building such a solution requires optimizing resources, balancing and trading off latency, privacy, bandwidth efficiency, and accuracy for the target application. This project presents a **prototype** edge-cloud video analytics distributed pipeline and Streamlit dashboard for monitoring. I designed this solution to be scene-aware with low bandwidth dependency and low latency to achieve s detection quality (count and accuracy). The orchestrator manages pipeline modules by preprocessing and filtering frames and conducts on-edge fast object detection to decide whether to offload the frame to the cloud. The system filters frames using a motion detector and performs selective cloud offloading to demonstrate these trade-offs. The system concurrently records per-frame metrics (latency, bytes offloaded, etc.), and the dashboard displays these in near real-time. Experiments show that the heuristic filter reduces processed frames by ~55%, yielding substantial reductions in edge processing, cloud offloading and bandwidth usage. The edge-only processing reduced end-to-end latency by ~50% compared to a cloud-only processing, while maintaining comparable detection quality.

2. Architecture

This solution utilises an edge-cloud setup, designed for motion/change awareness, low bandwidth dependency, low latency, and near real-time detection. The edge decodes the video, pre-processes frames (resize and greyscale conversion), and filters them using a heuristic change/motion detector.

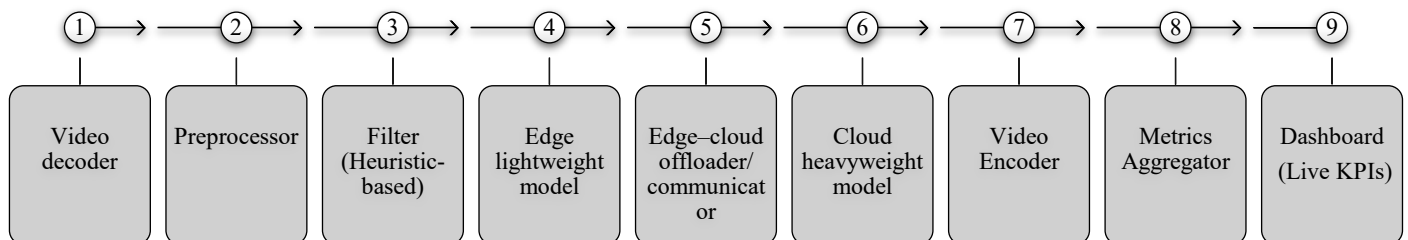


Figure 1: Real-time video analytics architecture incorporating and orchestrator that decodes videos into individual frame, performs preprocessing and conducts on-edge object detection and decides whether to offload frames to the cloud server. An auxiliary live dashboard

A lightweight on edge object detector conducts inference and selectively offloads frames to the cloud. The cloud runs inference using a more capable and more accurate model through a Flask API and returns results. Concurrently, a small metrics aggregator collects and writes metrics as JSON snapshots. This JSON file is used by a Streamlit dashboard to display live KPIs. Figure 1 shows system's components including:

- Video decoder: Breaks down raw video stream into individual frames.
- Preprocessor: Resizes and converts frames to greyscale.
- Heuristic frame filter: Detects change/motion using cheap pixel intensity difference between two greyscale frames and drops redundant frames.
- Edge model: A lightweight object detector (YOLO) that decides whether to offload the frame to the cloud.

- e) Edge–cloud offloader: Converts/encodes the frame into JPEG bytes and transmits them to the cloud when required.
- f) Cloud heavyweight model: A larger, more accurate cloud model makes inference on the received request and returns detection results.
- g) Video encoder: Handles edge-side video output, uses predictions to annotate frames, and writes the annotated output or displays it live.
- h) Metrics aggregator: Concurrently records per-frame telemetry/metrics (latency, bytes sent), calculates analytical metrics (e.g., cloud avoidance ratio), and writes them as per-frame snapshots.
- i) Dashboard (auxiliary module): A Streamlit UI that visualises edge, network, cloud, and content metrics in near real-time for monitoring and analysis purposes.

3. The design

The main design objectives included: **Low Latency:** The speed of detection translates directly to timely decision-making, which is very vital in application such as threat detection and alert. **Bandwidth efficiency:** Bandwidth can be a scarce resource in many remote and constrained environments. (e.g., military compounds), video analytics involves continuous processing of high volume of visual data from multiple sources/sensors, heavy reliance on a cloud server requires offloading monolithic visual data through network, which needs state of the art network infrastructure with high capacity bandwidth, in this solution I strive to reduce reliance on cloud server and preserve bandwidth. **Privacy preservation** is another key objective, security related video analytics applications usually demand high privacy due to the nature of the location under surveillance such as military bases, I strive to reduce the exposure and attack surface raw data by enabling the system to make local on edge decisions and conducts selective offloading to the cloud server when required. **Prediction quality/accuracy**, in application such as threat detection and warnings quality of detection is as important as timeliness, as missing a threat source can result in huge losses. I aim to balance quality of detection (accuracy and count), and cost (latency and bandwidth). **Lastly, Monitoring and observability** is an important aspect for this prototype, the dashboard provides continuous real-time monitoring of performance metrics and scene content, which aid evaluation and further processing, system debugging, and threat detection. The design and implementation of this system is based on the following assumptions:

- Input video format: The input video either streamed live or prerecorded is in standard format such as MP4, that can be decoded using OpenCV '*cv2.VideoCapture*' without additional custom logic or code.
- Mounted static camera or have steady controlled motion: Video source should be stationary with controlled steady movement if any (like CCTV cameras) this ensures that the analytic pipeline can take advantage of the edge intelligence (heuristic filter and edge detection).
- Limited but non-trivial edge compute power: I designed the edge device to conduct preprocessing, heuristic filtering and fast detection utilizing a lightweight YOLO model, these processes require a non-trivial compute power at the edge.
- Native video resolution: The original video frames have enough resolution/spatial detail, so resizing (e.g. to 640px width), does not distort or remove important objects' information from the frame.

- Audio information: My design completely omits and ignores audio data and assumes that it's not required for the analytics.

Design Decisions and Rationale: Design decisions and choices were made based on the objectives (low-latency, bandwidth efficiency, privacy, detection quality, and observability) and taking into consideration the real-life nature of the application. The following list is the pipeline's main modules, their responsibilities, design rationale, and link to the core objectives.

- a) Video Decoder (Edge): The configurable frame skipping reduces the workload, speeds up computation, decreases network load, and ultimately improves latency and bandwidth efficiency.
- b) Preprocessor (Edge): Smaller frames are faster to compute and cheaper to transmit over the network. The grayscale versions of frames are very cheap to compute when assessing the heuristic change/motion detection. This aids latency and saves bandwidth.
- c) Heuristic Filter (Edge): Filtering redundant frames early in the pipeline minimizes the need for unnecessary inference and potential transmission over the network, which improves overall efficiency and saves bandwidth.
- d) Lightweight Edge Model: This enables fast local inference and decision-making and avoids offloading sensitive data, preserving privacy, saving bandwidth, and reducing end-to-end latency.
- e) Offloading Module: JPEG compresses the image to a few bytes to be sent over the network, effectively improving bandwidth utilization, aiding privacy, and facilitating better prediction quality/accuracy from the edge server.
- f) Heavyweight Cloud Model: High-accuracy object detection model improves the system's overall prediction quality/accuracy and serves as a fallback for the local lightweight model. Furthermore, it helps scalability by serving multiple sources/sensors.
- g) Video Encoder (Edge): This module enables and facilitates real-time observability and situation monitoring, giving the human operator/ controller situational awareness, supporting decision-making.
- h) Metrics Aggregator: This module serves as the brain for monitoring and observability objectives. The per-frame metrics snapshots enable system monitoring, analysis, and debugging. They constantly feed the live dashboard with near real-time metrics to be displayed.
- i) Dashboard Edge /User: This module enables monitoring and observability of all metrics, i.e., edge, network, and cloud metrics, and shows content information, e.g., threat detection signal.

4. Algorithm and logic

The pipeline's orchestrator is based on the logic shown in Figure 2, which includes the following modules/components:

Frame generator: Receives and reads live or prerecorded video stream and decodes it into individual frames using `cv2.VideoCapture()`. This step has a skip interval as a configurable variable, which controls the number of frames to skip natively without any processing.

Preprocessing: Includes two steps; resizing the frame to a smaller size (has a configurable variable i.e. `resize_width`, default=640). The resized frames are then converted to greyscale using `cv2.cvtColor()`, a single colour channel with pixel values ranging from (0/black — 255/white).

Heuristic filter: A vital component of this pipeline. As illustrated in Figure 3, this filter is computationally cheap as it uses the grayscale frame to check whether the frame is interesting by calculating a change score (0–255) based on the mean absolute difference in pixel intensity between the current frame and its predecessor using `cv2.absdiff()`. This change score serves as a motion indicator. It compares each frame's change score to a configurable heuristic threshold to decide based on this logic:

- Change score $\geq$ heuristic threshold: frame is interesting (continue processing), return (True, the change score).
- Change score $<$ heuristic threshold: return (False, the change score).

If False, drop the frame (no further processing). If True, the frame is interesting and significant change has been detected. Consequently, continue processing the frame.

Edge model inference: To improve efficiency, the orchestrator lazily loads the model using Ultralytics YOLO class (default `yolov8n.pt`) and conducts an asynchronous background warmup using `threading.Thread()` to avoid blocking the pipeline. For each interesting frame, the orchestrator conducts on-edge object detection using the `_detect_edge()` method and measures inference time in milliseconds. The results are converted from a single Ultralytics object into a structured list of dictionaries using the `parse_detections` function and are used to annotate the frame. A wrapper method `edge_model_decision` extracts per-detection confidence scores and implements a gateway logic:

- If no detections (empty conf list) or $\max(\text{confidences}) < \text{edge_conf_threshold}$: offload to cloud.
- If $\max(\text{confidences}) \geq \text{edge_conf_threshold}$: accept the detections and annotate the frame locally and prepare it as `display_frame` for output.

Edge confidence threshold range is (0 – 1), and it's configurable through the application CLI. All other exceptions, e.g., edge model unavailability, result in an empty response, so the orchestrator can handle them gracefully by routing the frame to the cloud server.

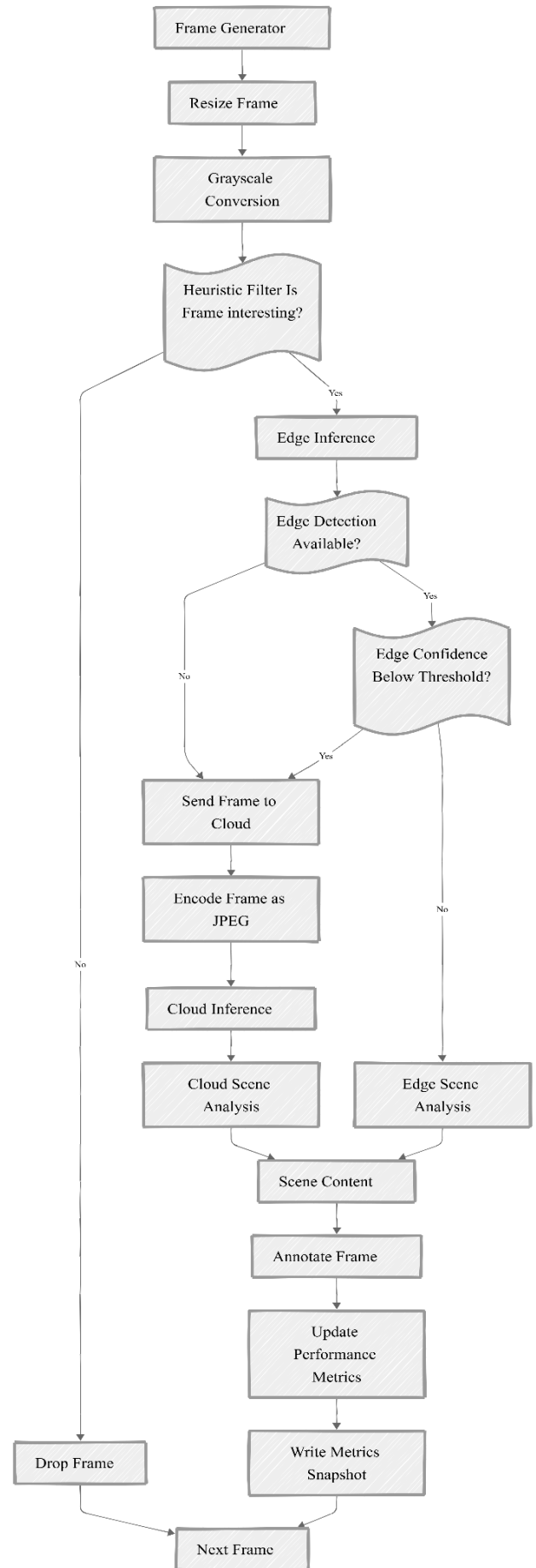


Figure 2 Algorithm/ logic flowchart of the video analytics pipeline illustrating frame preprocessing, heuristic filtering, edge inference, cloud offloading and metrics aggregation and display

Offload frames to the cloud server: Once the decision has been made to offload a frame to the cloud server, the orchestrator encodes the resized frame into in-memory JPEG bytes using `cv2.imencode()`. The orchestrator runs a bytes counter to track bandwidth usage. The frame bytes are POSTed as a multipart payload to the cloud server endpoint using `requests.post()`. At this point, metrics such as latency in the form of request round-trip time in milliseconds and count of the frames sent to the cloud are continuously measured. All errors and exceptions related to network calls are handled gracefully so the pipeline continues without crashing.

Cloud Inference: The cloud server is implemented as a Flask HTTP endpoint (POST/infer), it accepts the multipart frame payload request, conducts a more accurate object detection in using heavyweight YOLO mode (default `yolov8s.pt`) and compute inference time. The server lazily loads the model allowing it to be initialised on demand and reused across multiple requests to avoid unnecessary resource allocation on the cloud. The results are converted from single Ultralytics result object into a structured list of detection dictionaries using the `'parse_detections'` function. On successful transmission, the cloud server returns a JSON response, the edge use it to annotate the frame as part of the output.

5. Performance Metric

I designed the pipeline to collect key metrics related to edge, cloud, and network processes as well as video content information. These metrics are consciously collected as per-frame snapshot and recorded to "output/metrics_history.json" using `'json.dumps()'`, For further details on the analytical metrics refer to Table 1 hereunder. The basic foundational metrics describing system activity include:

- **Frame_index:** The sequential frame identifier assigned to each frame indicating its position within the video stream.
- **Timestamp(s):** The time(s) in the video stream at which the frame was captured, as seconds from the start of the video.
- **Total Frames Seen:** The total number of frames extracted from the video stream.
- **Frames Heuristically dropped:** The total number of frames dropped by the heuristic filter.
- **Frames Send to Cloud:** Shows how often cloud offload occur.
- **Edge Inference Time:** The per-frame inference time (ms) on the edge.

Cloud Inference Time: The per frame-inference time (ms) measured on the cloud server.

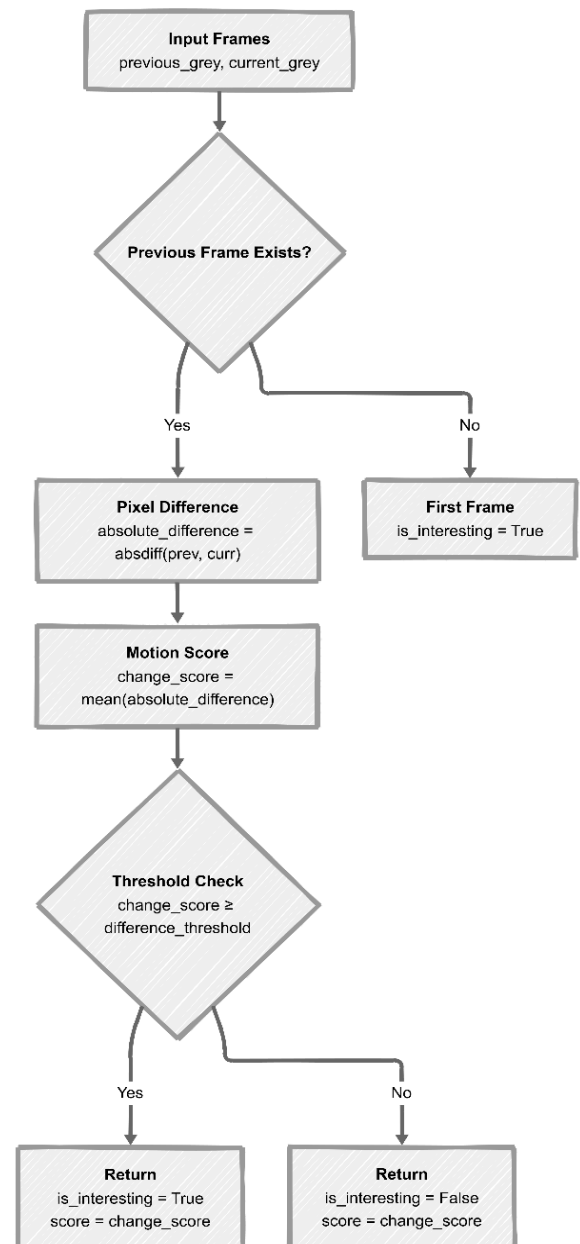


Figure 3 The heuristic filter algorithm showing how a frame is considered interesting based on pixel intensity difference with its predecessor.

Table 1: The high-impact analytical metrics, their meaning, rationale and how they are computed:

Metric	Meaning and Rationale	Computation
Edge Heuristic Drop Ratio	The ratio of frames deemed redundant by the heuristic filter. It quantifies the intelligence and effectiveness of the heuristic filter in eliminating redundant workload early, effectively improving the efficiency of the entire system.	Heuristically dropped frames $\div$ Total frames processed
Cloud Avoidance Ratio	The ratio of cloud communication and inference avoided due to edge intelligence. This metrics quantifies the edge intelligent design and directly corresponds to reduced end-to-end latency and bandwidth saving.	$1 - (\text{frames sent to cloud} / \text{total frames seen})$
Avg Edge Inference Time (ms)	The average time for the edge model to make inference. Describes overall speed of edge inference.	Mean of all the per-frame edge inference times
Avg Cloud Inference Time (ms)	The average time for the cloud model to make inference. Describes overall speed of cloud inference.	Mean of all the per frame cloud model inference times
Avg Round-Trip Latency (ms)	The average latency/round-trip time measures the end-to-end delay for cloud offloading (includes network latency and cloud inference time).	The mean of all cloud round-trip times (between frame transmission and response reception)
Bandwidth/Network Usage	Measures the total data transmitted from the edge to the cloud, indicating network bandwidth consumption.	The total cumulative megabytes offloaded to the cloud server.
Avg Detection Confidence and Total Objects Detected	This is the per-frame average detection confidence and the total count of object detected. Together they serve as an indicator of the quality of the object detection.	The mean of all per-frame detection confidences. And the count of the objects detected.

Application Scenarios: This prototype is configurable, adaptable, and has the potential to be applied in a wide range of scenarios including perimeter surveillance, traffic monitoring, smart cities, and intrusion detection. Configurability of the heuristic filter threshold and edge model confidence threshold, enable the user to tune the pipeline and trade detection quality/ accuracy for cost (bandwidth and latency). The default YOLO models used on the edge and cloud server are also configurable and can be changed to another YOLO alternative, e.g. a variant that specialises in weapon detection.

6. Dashboard

I implemented a Streamlit-based dashboard to display the key performance metrics and scene content. This dashboard automatically and continuously reads the per-frame JSON metrics history file, updated by the orchestrator, and displays a compact set of KPIs and charts, including Timestamp (s), Heuristic Drop Ratio, Cloud Avoidance Ratio, and so on. Furthermore, the dashboard includes time-series plots displaying trends over time, e.g, edge inference time, cloud inference time, etc. The dashboard also contains an optional raw JSON data display in an organised and nice table display.

7. Implementation

This solution was implemented as a two-tier edge-cloud video analytics system using Python in approximately 1,200 total lines of code (LOC). The edge node decodes the video streams using *OpenCV*, perfumes cheap-to-compute preprocessing, applies heuristic filtering and runs a fast object detection using Ultralytics YOLOv8n as a default. The edge conducts selective frames offload, in which frames are compressed as JPEG bytes and sent through HTTP to a

Flask cloud API, that run more powerful model (Ultralytics YOLOv8s as default). I designed the edge node to annotate frames locally (utilising NumPy/OpenCV) and writes or live display the annotated video. Concurrently, metrics are calculated and written to `'output/metrics_history.json'`. I build a *Streamlit* dashboard to continuously read the metrics JSON file and visualise the performance KPIs, and scene content, in near real-time. Core Python libraries used include `'opencv-python'`, `'ultralytics (YOLOv8)'`, `'flask'`, `'requests'`, `'numpy'`, `'pandas'` and `'streamlit'` the complete detailed list of dependencies and packages is available in the project root `'requirements.txt'` file.

Implementation components — The project root contains four main logical sections that correspond to edge, cloud, common processes, and dashboard, the contain helper function, alongside the following core components:

- **Orchestrator (edge/orchestrator.py):** This is the primary pipeline coordinator that handles the end-to-end video analytics journey; it receives the raw input using `'video_frames_generator ()'` conducts preprocessing (`resize_frame()`, `convert_to_grayscale()`), applied heuristic filter (`is_frame_interesting()`), runs the edge model gateway (`EdgeModel.edg_model_decision()`), offloads JPEGs to the cloud server when required (`encode_to_jpeg_bytes()` and `feed_cloud_jpeg()`), writes the per-frame metrics snapshot using `'write_metrics_snapshot()'` annotate frames (`annotate_frame()`) and produces the final annotated video output.
- **Edge model (edge/edge_model.py):** Lightweight YOLOv8n on-cloud model, it has lazy import and background warm-up features to avoid blocking the main pipeline. It conducts fast objects detection and returns detection conferences; based on these confidences the pipeline decides whether to accept these detections as final of offload to the cloud.
- **Cloud server (cloud/server.py):** This is a Flask API that accepts requests sent by the edge in the form of multipart JPEG, it uses a more capable model (YOLOv8s) to conduct object detection and uses helper functions such as `'parse_detections()'` and `'detect_intrusion()'` to parse data and create the scene content in a JSON response.
- **Common utilities (common/):** Contains common logic utilised by both cloud and server these includes methods such as `'annotate_frame()'` to annotate frames, `'write_metrics_snapshot()'` automatically updates the metrics history file (`output/metrics_history.json`), and `'detect_intrusion()'` to produce the scene content.
- **Dashboard (dashboard/dashboard.py):** This is a *Streamlit* app that continuously reads the metrics history (`output/metrics_history.json`) and displays system KPIs and scene content.

8. Experiments and Results

Objective – To demonstrate the prototype operability and quantify the value of edge intelligence (heuristic filter and edge detection) in the trade-off between cost (bandwidth and latency) and detection quality/accuracy.

To stress-test the pipeline and effectively measure trade-offs, I use a busy-scene (multiple pedestrians and vehicles), this yields more detections, cleaner metrics signal, for a thorough analysis.

Compute Environment – I conducted these experiments on two controlled and secure virtual machines, each with 8 vCPUs (Xeon 8562Y+), 15 GB RAM, No GPU, AlmaLinux 9.7, and python 3.9.25.

Experiment 1: Edge-only vs. Cloud-only — I ran two analytics queries for the same video (`data/experiment_sample.mp4`), isolating where inference takes place. In both cases: `--skip_interval=0` (process all frames), and `--heuristic_threshold=5.0` (default).

- **Edge-only run:** To prevent any offload even if the edge reports no detections, I changed the gateway logic to be (if not confidences_list: don't send_to_cloud). And set edge_conf_threshold=-1 (below the possible minimum), so $(\max(\text{confidences}) < -1)$ is always false, i.e., don't sent to the cloud.
- **Cloud-only run:** I kept the default cloud gateway logic (if not confidences_list: send_to_cloud). And used the CLI to set edge_conf_threshold=1.1 (above the maximum possible), so $(\max(\text{confidences}) < 1.1)$, is always True, i.e., always send to the cloud.

Post these two runs I converted the raw JSON history into DataFrame and extracted the final snapshot metrics.

Table 2 shows the cumulative metric as per the last metrics snapshot.

Run	Total_Frames_Pro cessed	Frames_Sent_to_clo ud	Heuristic_Drop_R atio	Cloud_Avoidance_R atio	Bandwidth_Used (MB)
Cloud-only	5108	2278	0.55	0.55	117.75
Edge-only	—	0.00 ↓	—	1.00 ↑	0.00 ↓

Cumulative Metrics Summary

- **Cloud-only:** Frames are either dropped by the heuristic filter or offloaded to the cloud (heuristic drop ratio=cloud avoidance ratio). ~117 megabytes were transmitted.
- **Edge-only:** All frames were processed locally (cloud avoidance ratio=1), with no bandwidth consumption.

Cost (Latency and Bandwidth) vs. Detection Quality

Trade-off — To compare cost and detection quality across the two runs I aligned the two metrics datasets using 'frame_index' as the common denominator and from the two dataset I extracted the per-frame:

- Objects detected count. Average detection confidence/accuracy.
- Latency, in cloud-only run this is the round-trip time (ms), in the edge-only run this is equal to edge inference time (ms). Figure 4 plots frame index (X-axis) against these metrics (Y-axis) and demonstrates a trade-off.

Objects detected — The plot's overall trend shows that the cloud consistently detects more objects per frame than the edge. This indicates higher detection sensitivity from the cloud model.

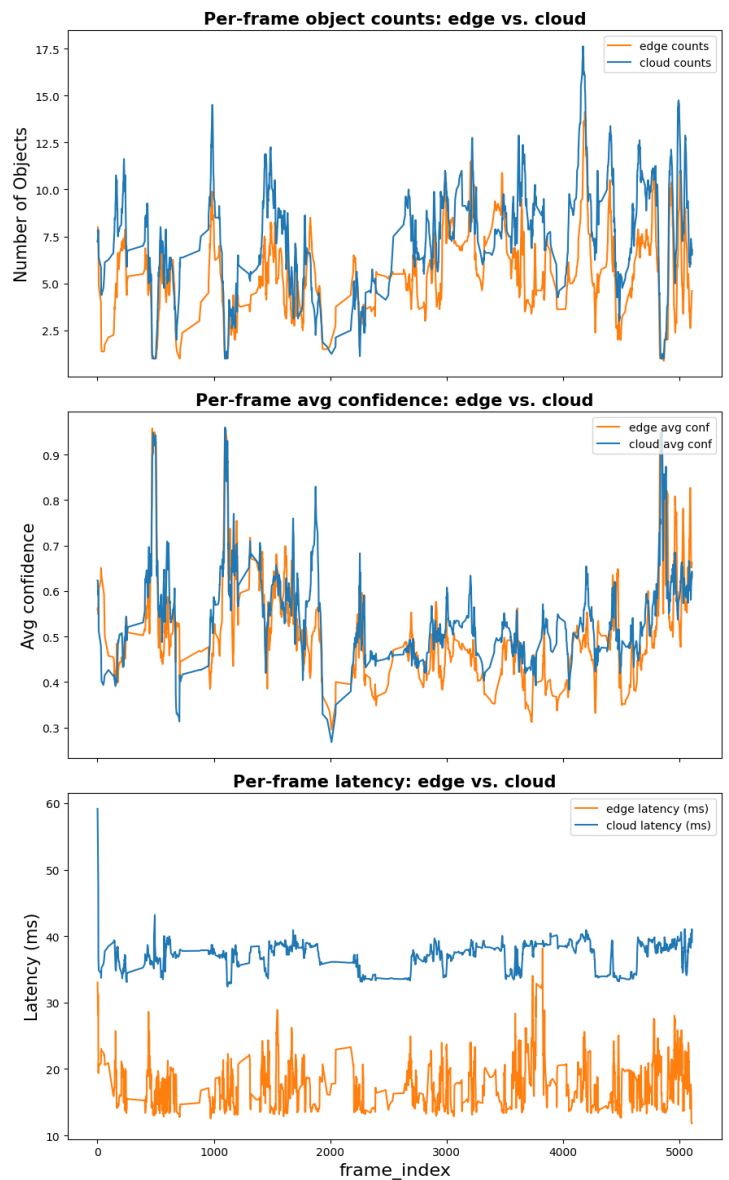


Figure 4 Smoothed (8 frames rolling mean) per frame time-series of count of objects, average confidence, and latency.

Average confidence (proxy for accuracy) — The plot shows that cloud also exhibits a slightly higher per-frame mean confidence compared to the edge. This indicates that the edge model has comparable confidence on average when it detects objects, but the cloud remains slightly better.

Latency — Edge latency is significantly lower than the cloud latency, as cloud incurs transmission overhead. On average edge latency is ~17 ms, and cloud latency is ~37 ms, making cloud inference ~2× slower.

Trade-off — A trade-off is empirically observed: the cloud provides modest but noticeable better detection quality (more objects with slightly higher accuracy), at the cost of a significantly higher latency (~2×). Edge inference offers faster response while maintaining comparable detection quality.

Caveat — I use the per-frame average confidence as a proxy for the “detection quality”. Confidence scores are not calibrated across model sizes (small vs. large YOLO), higher confidence does not strictly imply higher true accuracy. Therefore, I incorporated per-frame objects count to contextualise the ‘detection quality’.

Experiment 2: Heuristic Filter Efficiency — To evaluate filter’s impact and effectiveness, I ran two analytics queries (filter OFF/ON) on the same video (*data/experiment_sample.mp4*), with `--edge_conf_threshold=0.70` (default), and `--skip_interval=0`.

Table 3 shows the cumulative metrics of the two runs.

Run	Total_Frames_Processed	Heuristically_dropped	Heuristic_Drop_Ratio	Processes_on_Edge	Sent_to_cloud	Bandwidth_Used (MB)
Heretic OFF (threshold=0)	5108	1*	0.00	2519	2588	133.12
Heuristic ON (threshold=5.0)	—	2830 ↑	0.55 ↑	1518 ↓	760 ↓	39.39 ↓

**Why 1? Heuristic filter seeds the first frame, therefore there will always be 1 frame ignored/dropped frame due to the initialization logic (refer to experiments Jupiter notebook for full explanation)*

Heuristic filter OFF (threshold=0) — The heuristic drop ratio is zero (no filtering), this resulted in a high workload, every frame is processed, either by the edge or the cloud.

Heuristic Filter ON (threshold=5.0) — For greyscale frames pixel intensity range is [0–255], the default threshold (5/255 of the range) corresponds to a tiny pixel-intensity change, i.e., dropped frames are visually and statistically near-identical to their predecessors. With the filter ON, Heuristic drop ratio=55%, this corresponds to an early reduction in overall workload/frame for the entire pipeline by 55%, without losing vital information. This created cascading gains: ~40% less edge workload, ~70% of bandwidth savings, and ~71% less cloud workload. See Figure 5.

Overall, edge intelligence (filter and detection) empirically improves pipeline’s efficiency. The heuristic motion filter eliminated redundant

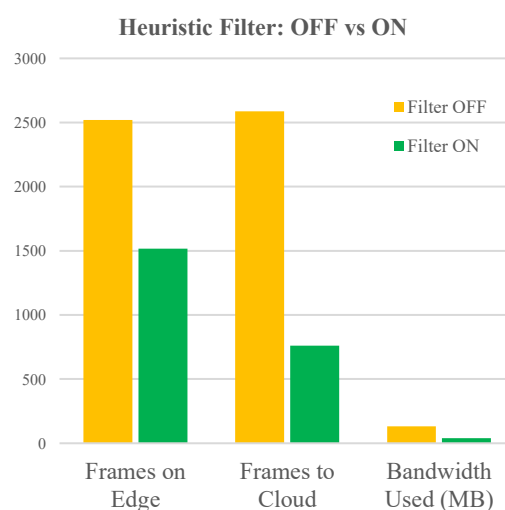


Figure 5 Workload (bandwidth and frames count) comparison between heuristic filter ON/OFF.

workload early reducing cost (compute and bandwidth). The lightweight edge detection reduces latency while maintaining reasonable detection quality (count and accuracy).

9. Limitations and Future Improvements

The current prototype focuses on design, implementation and operability. Real-world production deployment must be optimized for efficient scalability (multiple sensors generating frames simultaneously) and account for network connectivity issues. Therefore, this prototype can be improved by incorporating an asynchronous (as opposed to the currently synchronous) request queuing to avoid loop stalling at scale (multiple cameras) and when network connection issues occur which increases transmission latency. This will improve scalability and production robustness. Furthermore, production analytics would benefit from object tracking across frames to improve counting stability and object identification.

10. References

- Ao, L., Izhikevich, L., Voelker, G. M., & Porter, G. M., n.d. Sprocket: A Serverless Video Processing Framework.
- Jiang, J., Ananthanarayanan, G., Bodík, P., Sen, S., & Stoica, I., n.d. Chameleon: scalable adaptation of video analytics.
- Ultralytics, n.d. Ultralytics YOLO Documentation. Available [at](#).
- Streamlit, n.d. Streamlit documentation. Available [at](#).